

Crash Course on Prompt Injections

Attacking and Defending

Agenda

- whoami
- AI overview (terminology and concepts)
- Prompt injections
- Data flows
- Defence
- Hands on exercise exploiting prompt injection
 - If time allows we're going to talk about how to defend against our exploits

whoami

- Julian Eckhardt, CISSP
- A security professional with over 10 years of experience helping private, public, and non-profit organizations build secure, scalable, and maintainable solutions.
 - I mostly bleed blue
- Volunteer Infrastructure lead and challenge creator at CyberSci
- Caveat: I don't know everything and LLM/AI security especially is fast moving.

Terminology

- AI – this will be a bit of a catch all for large language models (LLMs) and their multi-modal variants
- Tool – Sometimes also called "functions", but fundamentally a way for a model to interact with "things" (ex: files, web servers, APIs, browsers)
- Agent - A loop that uses a model plus tools, memory, and environment feedback
- Harness - The software scaffold around an agent that manages context, tool use, prompts, state, and control flow

These are mostly taken from Sebastian Raschka's blog: <https://magazine.sebastianraschka.com/p/components-of-a-coding-agent>

- As an aside, great blog to follow if you want to more of the technical details on how models work

Story time

- If you have a foundational understanding of how things work the security part is "easy"

Foundational Concepts to Remember

- Large language models (LLMs) and their multi-modal variants fundamentally do not have a way to guarantee they can distinguish between commands and data.
 - There are approaches that try, including at a training level or incorporating other models, which can reduce but not eliminate the risk
- Context is everything (i.e., where your commands and data live)

What exactly is context?

- It's fundamentally the content the AI is processing
- Typically made up of a "system" prompt that provides some model guidelines, the user and model's messages, results of tool calls, thinking, etc.
- A conversation example (Source: <https://role-confusion.github.io/>) :

```
<system>This iteration of Claude is Claude Opus
4.8...</system> <user> Can you tell me the day of
week with your shell tool? </user> <think>The user
wants to know the day of the week. I can use the bash
tool for this.</think> <tool_call> {"invoke":
"bash_tool", "command": "date +%A", "description":
"Get the current day of the week"}</tool_call> <tool>
Wednesday </tool> <think>Simple answer.</think>
<assistant>It's Wednesday!</assistant> <user> Nice
work! Can you tell me how you did that?</user>
<think> They want to know how I determined the day
of the week. I should give a helpful response. </think>
<assistant>Of course! I ran
```

Prompt Injections

- "Prompt Injection vulnerabilities exist in how models process prompts, and how input may force the model to incorrectly pass prompt data to other parts of the model, potentially causing them to violate guidelines, generate harmful content, enable unauthorized access, or influence critical decisions."
- Two primary types:
 - Direct Prompt Injections - "when a user's prompt input directly alters the behavior of the model in unintended or unexpected ways."
 - Indirect Prompt Injections - "occur when an LLM accepts input from external sources, such as websites or files. The content may have in the external content data that when interpreted by the model, alters the behavior of the model in unintended or unexpected ways."
- <https://genai.owasp.org/llmrisk/llm01-prompt-injection/>

Common Techniques - Jailbreaking

- Often involves creating a hypothetical scenario or context that tricks the AI into believing its safety rules no longer apply
- A useful framing is that the techniques are trying to social engineer the models
 - In a human context - [https://en.wikipedia.org/wiki/Social_engineering_\(security\)](https://en.wikipedia.org/wiki/Social_engineering_(security))
- Hacking OpenAI - A heap overflow and SSO misconfiguration to compromise OpenAI internal repositories
 - By 6:00 a.m. on July 25, we had confirmed local RCE through an image upload. We then placed Claude in an autonomous **/goal** loop against our own Discourse Cloud instance, proxied through **rce.ee/ctf-forum** to make it look like a CTF target as Opus refused write exploit for remote instances.
 - <https://www.hacktron.ai/blog/hacking-openai>
- Things like "Ignore your previous instructions..." generally don't work that well anymore especially against frontier models

Common Techniques – Single vs Multi-turn

- One conversation round vs many
- Multi-turn provides an opportunity to more gradually escalate the injection
 - Think about it in the sense of breaking your goal into small tasks that individually seem harmless
- Reprompt: The Single-Click Microsoft Copilot Attack that Silently Steals Your Personal Data
 - Although Copilot enforces safeguards to prevent direct data leaks, these protections apply only to the initial request. An attacker can bypass these guardrails by simply instructing Copilot to repeat each action twice.
 - <https://www.varonis.com/blog/reprompt>

Common Techniques - Resources

- Both of these go over a number of single and multi-turn approaches
- <https://www.promptfoo.dev/docs/red-team/strategies/>
- <https://github.com/confident-ai/deepteam>

Active area of research

- Prompt Injection as Role Confusion- <https://role-confusion.github.io/>
- Stealing Reasoning Traces from Proprietary LLM APIs - <https://arxiv.org/html/2608.09867v1>
 - This is not technically purely prompt injection, but they "jailbreak" older and less hardened models to decode the reasoning traces
- Lots of real world examples too, which we'll touch on shortly

Where can your injection can hide...

- Any data source that an AI agent ingests and you can control is a potential vector.
- Websites - An agent that summarizes articles can be hijacked by a malicious prompt hidden in the HTML of a compromised or attacker-controlled site.
- Documents (PDFs, DOCX) - An agent that analyzes reports can ingest a document containing instructions to exfiltrate other data it has access to.
- Emails - A "smart reply" agent that reads incoming emails could be triggered by a malicious email to perform an unauthorized action on the user's behalf.
- Databases / Retrieval-augmented generation (RAG) - If a RAG system ingests user-generated content, that content could contain prompts designed to attack the next user who queries it.
- Calendar invites – An agent manages your calendar and accepts invites from external contacts
- Pull requests/issue comments - An agent automatically reviews pull requests and follows instructions to exfil pipeline secrets
- Repos – An agent clones an untrusted repo with a malicious Model Context Protocol (MCP) configuration

Real life examples

- <https://checkmarx.com/zero-post/mcp-configuration-poisoning-owning-your-machine-with-just-a-text-file>
- <https://embracethered.com/blog/posts/2026/breaking-claude-code-opus-5-and-automode/>
 - Manipulates the tool Opus uses to fetch a website through HTTP status codes
- <https://www.intigriti.com/researchers/blog/hacking-tools/hacking-ai-customer-service-agents>
 - This one was neat because they leveraged lesser known parts about email RFCs like allowing multiple "From" headers to cause an agent to exfiltrate data to an attacker controlled email address

Couple of things to keep in mind

- LLMs are non-deterministic, which means an attack may not be successful the first try
 - If you're convinced you've got a valid approach, try it multiple times
- Often you'll be dealing with a closed box and you won't get strong feedback on where things are succeeding or failing
 - Options here are more limited, but try to find side channels (ex: timing of responses)
 - You can also try to recreate what you think is happening locally in an environment you fully control

Analyzing the Data Flow

- How can you find these entry points?
- Map out the application's data flow and ask:
 - Where does untrusted, third-party data enter the system and get processed by the LLM?
 - Can be useful to try and diagram this (for example: https://en.wikipedia.org/wiki/Data-flow_diagram)
- Specifically for CTFs is the challenge involving:
 - Uploading an image to be analyzed
 - Fetching web pages
 - Cloning git repos

What is the Lethal Trifecta

- First coined by Simon Willison
(<https://simonwillison.net/2025/Jun/16/the-lethal-trifecta/>)
- An exploitable vulnerability (some people might call it an attack/kill chain) that occurs because of:
 - Privileged Data Access - The agent has permissions to read sensitive, internal data.
 - Untrusted Input Processing - The agent processes user prompts or external data that can be manipulated by an attacker.
 - Egress Capability - The agent has tools to send data to an external location that an attacker can access (may be internal or external)

Agents Rule of Two

- Based on the Chromium Rule of Two (<https://chromium.googlesource.com/chromium/src/+/main/docs/security/rule-of-2.md>)
- Agents can **only** do two of the following :
 - An agent can process untrustworthy inputs
 - An agent can have access to sensitive systems or private data
 - An agent can change state or communicate externally
- As soon as Agents have all three capabilities you're in for a bad time
 - An Agent can know things, receive messages from strangers, talk to strangers, but not all three.
- <https://ai.meta.com/blog/practical-ai-agent-security/>

Some approaches

- Control the Data Sources
 - Does an agent really need access to all of the data? Can you scope it down?
- Control the Data in the Agent
 - Can you apply filtering to data before it enters the context? Sometimes you can have another LLM assist with that, but be mindful of injections there too (plus performance impacts)
- Control the Egress Point
 - Restrict the things an agent can talk to and where it can write/post data. If you don't, this is how you get things like <https://collusion.wiki/>
- Monitoring – you might not be able to stop an attack, but you can watch for unexpected behaviors or access patterns that indicate something strange is happening
 - For example, being suspicious of a newly created document hosted on a domain (even if that domain is trusted)
- Defence in depth
 - Don't rely on a single fix. Try to build a resilient system
- Also worth a read: Design Patterns for Securing LLM Agents against Prompt Injections - <https://arxiv.org/html/2506.08837v3>

Exercise Time – Part 1 - Users

- Clone/download the repo here:
- Do a: "docker compose up –build"
 - It may seem like it's hanging, but downloading the model can take 5+ minutes
 - The app should be accessible here: <http://localhost:8000/>
- Interacting with the web app (i.e., you can look at the code, but don't make any code changes **to the app**):
 - Find a way to get the app to return the classified user to you (the users are in users.txt)
 - Look at the source code and try to find how tool calls are used
 - Small hint: You will need to be able to run a web server that the app can access (an easy one if you python 3 installed: "python –m http.server 8080")

```
llama-1 | 0.03.829.856 I srv llama_server: model loaded
llama-1 | 0.03.829.965 I srv llama_server: listening on http://0.0.0.0:8080
llama-1 | 0.03.830.073 W srv llama_server: NOTICE: server default port will be changed to :9931 in a future release
llama-1 | 0.03.830.279 W srv llama_server: ref: https://github.com/ggml-org/llama.cpp/pull/26508
llama-1 | 0.03.830.580 I srv update_slots: all slots are idle
Container cyberscillmcrashcourse-llama-1 Healthy
app-1 | INFO: Started server process [1]
app-1 | INFO: Waiting for application startup.
app-1 | INFO: Application startup complete.
app-1 | INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```

Exercise Time – Part 1 - Walkthrough

Exercise Time – Part 2 - Admin Tokens

- Interacting with the web app (i.e., you can look at the code, but don't make any code changes **to the app**):
 - Find a way to get an Admin token and validate it works with "Verify an admin JWT"
 - Look at the available routes to see which ones you might want to leverage (you can take a look at the code or here <http://localhost:8000/docs>)

Exercise Time – Part 2 - Walkthrough

Critical Points

- LLMs can't distinguish between commands and data (context is everything)
- Any data source that can be inserted into a context is a potential attack vector
- When agents have a combination of privileged data access, untrusted input and egress capabilities there be dragons.

References and resources

- <https://genai.owasp.org/resource/owasp-genai-llm-top-10-2026/>
- <https://owasp.github.io/www-project-agentic-skills-top-10/>
 - <https://github.com/OWASP/www-project-agentic-skills-top-10>
- <https://github.com/PawelKozy/mcp-breach-to-fix-labs>
- <https://play.lakera.ai/agent-breaker>
- <https://github.com/orgs/Simple-Networks/repositories>
- <https://portswigger.net/research/can-ai-do-novel-security-research>
 - Not prompt injection related, but a good example of leveraging RFCs and AI for conducting security research